

從實體問題，到 AI 學習系統

A 國中 | Arduino 智慧遮雨棚

我負責 Arduino 接線、程式與 App Inventor 手機控制。這是我第一次把程式從螢幕上的運算，真正帶到感測器、馬達與實體裝置上，也因此開始接觸軟硬體整合。

2023 IEYI 臺灣選拔賽

國中組便利生活類 | **銀獎**

► 延伸資料 | 發明展資料 ↗

B 高一 | FBref 球員數據分析

自學 Python、pandas 與爬蟲。單一球員成功後擴大請求，IP 被限制；理解 Rate Limiting 後加入請求間隔。即使程式沒有 bug，不了解外部系統，整個系統仍可能失敗。

球員戰術儀表板 (近期狀態分析)				
球員	近期進球率 (GA)	防守組合指數 (XkinoBlocks)	近5場傳球成功率 (%)	近5場黃牌數 (Yell)
Ruben Dias	11	11	94.2	25
João Cancelo	10	10	87.6	22
Kerem De Beure	9	9	74.7	23
Matty Keane	10	10	89.2	23
Bernardo Silva	12	12	82.5	25
Odion Eboah	11	11	68.2	25

▲ 球員近五場各指標比較

► 延伸資料 | GitHub 程式碼 ↗

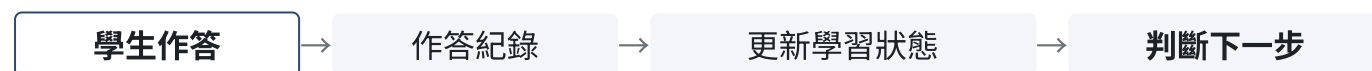
C 高二 | 自適應性輔助學習系統

► GitHub ↗ ► Demo ↗

高二時我想要參加育秀盃，指導老師要我們先找身邊真正存在的問題。我問在高職教數學的父親，他說同一班學生程度差距很大，老師難以在同一堂課替不同學生安排不同進度的練習與回饋——這成為我開發「自適應輔助學習系統」的起點。

我的分工 | 在三人團隊中，我主要負責教材進入系統、題目生成到學生作答回傳的資料主線，以及 Code-as-Content 的程式題生成；後續的 AST Healer 研究，也從這一部分延伸出來。

① 學生答完之後，下一步往哪走？



已掌握 → 下一題／下一技能

尚未掌握 → 找出弱點與前置概念 → 補強、AI 提示 → 推薦練習 → 再次作答 ↻

② Code-as-Content | 把「出題程式」當成題庫內容

題庫保存題型與教材內容，由語言模型產生 Python 出題程式；執行時帶入不同參數，同一支程式就能反覆產生不同題目與答案。



學生端 | 依章節與技能練習・AI 提示與補強・作答紀錄・後續推薦



教師端 | PDF／Word 教材匯入・班級管理・技能結構・查看學習紀錄

2026 扶輪盃全國高中 AI 競賽
高中組第一名

早期原型

第 23 屆育秀盃
全國高中職 AI 應用類金獎

系統完整成果

系統真正開始運作後，我開始把注意力放到另一個問題：程式能不能穩定、可驗證地運作。

系統維運問題，成為研究起點

A 研究起點 | 大型模型的成本與延遲

Code-as-Content 每次出題都要呼叫大型模型，token、API 成本與延遲持續累積（Gemini API 10 月峰值約 1,560 元）。改用本地小模型後，卻發現它生成 Python 並不穩定。

怎麼讓小模型少做一點，卻做得更可靠？

B1 | 生成前：哪些工作可事先抽離？

Scaffolding / Domain Functions

先整理工具、參數來源與呼叫順序，降低模型自己規劃的負擔。

B2 | 生成後：哪些錯誤可用規則修？

AST Active Healer

只修固定規格能唯一判斷修法的局部錯誤。
不能確定修法，就不介入。

C 初版驗證

41.00 → **94.39** → **97.41**
Bare + Scaffolding + Full Healer

MCRI 為自建評估指標；97.41 是 MCRI 分數，不是通過率。三種模型都出現相同的改善趨勢。

D 研究歷程中的競賽、評審提問與轉折

校內科展
優勝

第 66 屆東區科展
優等
未晉級全國

第 25 屆旺宏科學獎
全國決賽 20 強
全領域 846 件

入圍之後
收到評審建議

① 測試資料獨立嗎？

→ 開發／測試分離、評估前
凍結規則

② Healer 修好了什麼？

→ 逐筆追蹤每個原始失敗

③ 自建 MCRI 夠當證據嗎？

→ 聚焦 Healer、改用 PASS
／FAIL

F Math16 重新驗證

► 旺宏決賽報告 ↗ ► 評審建議 ↗

16 道公開會考題 × 3 模型 × 4 Prompt × 5 seeds = 960 單元

4 題 | 規則開發
240 單元

凍結 Prompt 與
Healer 規則

12 題 | 保留題正式測試
720 單元 · 不讀答案 · 不跨組

G 正式結果

720
正式測試

342
原始失敗

42
Healer 修改

9
失敗 → 通過

2.63%
安全救援率

- 2.63% 與初版 MCRI 97.41 是不同指標，不能直接比較。
- 342 個失敗中，144 個（約 42%）是答案或演算法錯誤，本就不在 Healer 範圍。

H Healer 的邊界

能修 | 輸出格式 · 局部結構 · 資料流 · 固定
API 規格可判定錯誤

不修 | 數學答案 · 核心演算法 · 題意理解 ·
需重新推理的錯誤

重新驗證後，我開始注意一套 AI 軟體系統要怎麼分開處理模型能力、流程設計、程式修復與測試邊界，也更確定想往軟體系統、程式分析與測試繼續學。

系統做得越大，我越知道自己還缺什麼

A 三個需要靠大學補足的能力

① 演算法與資料結構

我可以閱讀程式、拆問題，也能把多個模組整合成系統；但遇到需要把問題抽象成演算法、設計資料結構，並穩定完成實作時，仍不夠扎實。

APCS 觀念題 **92 分／5 級**

實作 **135 分／2 級**

這個結果很直接地反映：我在閱讀、理解與拆解問題上較成熟，演算法與正式資料結構訓練仍需要補強。

② 大型軟體系統與維護

系統變大後，我開始碰到模組相依、資料流、版本修改、錯誤追蹤與測試的問題。很多做法是一路實作後摸索出來的，我想知道這些問題在正式的軟體工程與系統課程中，應該如何被更有系統地處理。

對應 | 軟體工程・計算機組織・作業系統・軟體測試

③ 程式分析與 AI 系統可靠度

Scaffolding 與 AST Active Healer 讓我開始碰到程式分析、模型能力邊界與測試設計，但目前很多理解仍是從專案與實驗一路摸索出來的。

想正式理解 | 程式如何被分析・哪些錯誤可以用規則處理・模型與程式如何分工・測試如何界定可靠度

B 把「我做過的」和「我需要學的」對起來

高中已經碰過

- 大型網頁系統
- 教材／題庫／作答資料流
- AI 模型串接・Code-as-Content
- 測試與驗證・AST 程式修復
- 真實學生使用



大學需要正式補足

- 資料結構・演算法
- 離散數學・計算機組織
- 作業系統・軟體工程
- 程式分析・AI 系統

C 為什麼是陽明交大資工

高中專案把我帶到的問題，集中在大型軟體系統、程式可靠度、測試、程式分析與 AI 系統。我現在需要的，是把這些靠實作摸索出來的問題，用正式的資料結構、演算法、系統、軟體工程與程式分析方法重新理解。

我希望在陽明交大資工完整的資訊工程訓練裡，把這些基礎一門一門補齊，再把學到的方法放回仍在使用的學習系統與 Healer 研究裡檢驗。比起學校名氣，我更在意這裡能接住我已經碰到的問題。

高中專案讓我先遇到了很多資工問題；進入大學後，我想把這些靠實作摸索出來的經驗，用正式的資工方法重新理解一次。

下一步：把高中已經開始的問題繼續做下去

第一階段

把靠經驗做出的系統重新理解一次

現在的問題 | 目前很多系統設計，是遇到問題後一路自學、實作、修改累積出來的。

要學 | 資料結構・演算法・離散數學・計算機組織・作業系統

帶回專案 | 重新整理教材、題庫與作答資料流・重新檢查模組邊界・改善資料結構與系統架構・建立更清楚的測試紀錄。也重新檢查目前每個設計選擇背後的理由。

第二階段

把「能跑」往「可靠」推進

現在的問題 | Code-as-Content、Scaffolding 與 Healer 已經遇到不同模型的能力差異、生成錯誤、修復邊界、評估設計與可重現性問題。

要學 | 軟體工程・軟體測試・程式分析・AI 系統・可重現實驗方法

帶回專案 | 建立固定實驗流程・擴大保留題測試・保留逐筆錯誤紀錄・分開模型錯誤、程式錯誤與流程錯誤・重新驗證 Scaffolding／Healer 的適用範圍

第三階段

回到真實學生

這套系統到底有沒有真的幫助學生？

高中階段已經做到 | 教材匯入・章節／技能結構・Code-as-Content・多元作答・數學判分・學習紀錄・適性學習循環・AI 助教・教師端／學生端・真實學生使用

大學後要做深 | 更完整的學生知識掌握模型・比較不同推薦策略・建立合理對照組・長期追蹤學習狀態・分開驗證「系統可運作」與「學生真的進步」

我想參加的比賽

大學後，我也想帶著這套系統繼續參加 InnoServe、經濟部智慧創新大賞，以及聯發科技「智在家鄉」。前兩項希望延續資訊系統與 AI 應用的方向；「智在家鄉」則希望把系統帶回花蓮，看看它能不能服務更多學生。

InnoServe

教育 AI／資訊應用

Best AI Awards

AI 技術與實際應用

聯發科技「智在家鄉」

花蓮在地教育應用

進入陽明交大資工後，我希望繼續做這套高中已經開始的系統，把現在碰到的問題帶進更完整的資訊工程訓練裡，一個一個往下解。